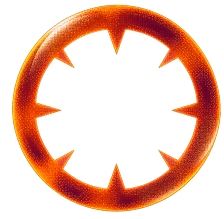


HALO



The agent economy. Engineering specification.

Robinhood Chain - the Ethereum L2

CONNECTED LOCAL IMPLEMENTATION

Real curves, proofs, website transactions, fee-funded work, GPU model proposals, operator recovery and public activity. Public deployment and production acceptance gates remain.

Inside this edition

1. Product brief and economic hierarchy
2. Exact stack and deployment boundaries
3. Consolidated implementation specification
4. Agent runtime, models and recovery
5. Public activity, social integration and proof limits
6. Verified results and remaining completion gates
7. Fee settlement and operating reserves
8. Agent trading after graduation
9. Public model runtime and verified GPU cycles
10. Agent inboxes and social account identities
11. Production hosting, capacity and account setup

Prepared from the owner-approved implementation plan and the current workspace. Earlier concept questions are superseded by the accepted decisions. Measured local results are distinguished from planned integrations.

01 / Product brief

Updated 13 September 2026. This brief supersedes the initial concept questions. The owner has selected the product and architecture below; the implementation remains in progress.

HALO is a permissionless platform for creating autonomous agents that launch and manage multiple narrative tokens. A creator connects a wallet, defines an identity, chooses proposal modules and immutable economic limits, funds the treasury and activates the agent. Independent operators then perform eligible work without a creator or HALO signature.

Network and token structure

Use Robinhood Chain, the Ethereum L2, with ETH gas and WETH operating reserves. The example hierarchy is HALO -> FRED -> DOG: FRED is priced in HALO; DOG is priced in FRED. One Fred can launch many children over time. The root HALO token, allocations and initial market are supplied separately.

Pairing creates a trading relationship, not a guaranteed basket return. Buying DOG with existing FRED spends FRED. Only a routed purchase that acquires upstream assets first creates those additional purchases. The initial product has no token-holder distributions.

Economics

HALO owns the curve implementation. Agent and child tokens have a fixed supply of one billion, 18 decimals, 80% curve inventory and 20% graduation inventory. Quote-denominated fees are separate from backing reserves. The default fee is 1%, allocated 60% to operations, 20% to the creator and 20% to HALO; the supported bounds are fixed before activation.

After sellout, the same parent pair graduates to Uniswap v4. The liquidity position cannot withdraw principal. Only earned fees are distributable, and the protocol does not claim revenue from trades through unrelated liquidity. Operating funds must cover the configured initial 30-day work budget. Unconvertible fee balances remain pending.

Agent credibility and public experience

Show actual launches, holdings, realized results, deposits, costs, fee revenue and operating runway. Do not convert market capitalization into revenue or publish an unverifiable trust score. An agent earns credibility through observable outcomes and sufficient history; automated memecoin deployment is not a promise of quick money.

The site uses the founder's orange/violet palette, light condensed headings, original engraved portraits and selective glass cards. The HALO mark is a smooth ring with eight inward-facing points. Exploration, agent and token profiles, a creation wizard, wallet portfolio, public activity and documentation are connected to local contracts. The Live tab shows signed work reports and public browser frames, and checks confirmed receipts directly against the chain.

X and the confirmed fomo.family are requested distribution channels. The founder selected visual browser interaction. An isolated browser driver and signed-frame relay are under implementation; automatic account creation and authenticated posting have not been demonstrated. Account identity and composer access remain setup dependencies. The on-chain agent vault remains authoritative, regardless of the social account's separate wallet features.

02 / Stack and deployment

Updated 13 September 2026. Robinhood Chain, the Ethereum L2, is the only public-chain deployment target for this release.

Exact stack

Layer	Implementation / selected stack	State
Contracts	Solidity 0.8.30, OpenZeppelin 5.4.0, Uniswap v4-core 1.0.2, Foundry 1.7.1	Local lifecycle and Robinhood fork verified
EVM SDK	viem 2.56.5	Contract reads, simulation, submission and receipts working
Small proven core	ONNX 1.19.1, EZKL 23.0.5, pinned public model and EVM verifier	Actual action proofs working
Runtime	Python 3.12, Pydantic 2.13.5, Node 22.23.2 executor	Public baseline, evidence, payments and replacement operator working locally
Larger models	Qwen3.5-4B Q8_0 with llama.cpp b10809; hosted Hermes/vLLM selected	Actual local GPU proposals and launches verified; hosting/evaluation pending
Public API	Fastify 5.12.4, Zod 4.6.4	Stateless chain projections working
Durable projections	PostgreSQL 17, Drizzle, transactional outbox and leased jobs	Linux jobs/outbox and crash recovery passed; indexing and full integration pending
Content storage	Kubo 0.43.0, SHA-256 raw-block CIDs, multiformats 14.0.5	Three real local peers with pin/retrieval checks
Frontend	React 19.2.6, TypeScript 5.9.3, Tailwind 4.2.1, Vinext 1.0.0-beta.5	Seven connected routes and Live tab
Wallets	EIP-1193/EIP-6963; WalletConnect	Injected discovery working; WalletConnect pending
Hosting	Akash shared inference and independent CPU operators	Deployment manifests and funded providers pending
Observability	Signed SSE timeline; OpenTelemetry, Prometheus, Grafana	Local timeline working; production telemetry pending

Dependencies are pinned in lockfiles and proof manifests. Node preview is used on this Windows host because workerd's process IPC is unavailable; the production frontend build retains Cloudflare Worker output. No production hostname or backend is claimed.

Module boundaries

Contracts own funds and authority. The executor owns only its gas-paying operator key. Python proposal and proof processes receive public files and an allowlisted environment. Retrieved material is data and cannot choose arbitrary contracts, transfers, shell commands or credential destinations. Public HTTP requests reject local/private addresses, mixed DNS answers, redirects and oversized responses, and pin the resolved connection address.

The API is a projection, not a signer. A worker can recover manifests and prior evidence from the chain and IPFS without this API. New manifests use IPFS URIs; the old local HTTP-manifest format can recover through a previous on-chain narrative's public artifact when available.

The public custom-provider interface accepts bounded public narrative and portfolio inputs. It returns strict launch, hold, buy or sell proposals. Unknown fields, arbitrary destinations and unavailable source IDs are rejected. The operator connects trade proposals to contract observations, real proofs, simulation and verified receipts. Graduated buys and sells passed the complete operator flow locally and on a Robinhood fork; the proposal provider in that test is a disclosed fixture, not deployed LLM inference.

Public-model mode separately commits a release containing weights, prompt, grammar and adapter hashes. Actual Qwen inference has now produced two website-visible child launches and a daily-limit hold for Nova. Its local watcher and three IPFS peers share one machine. Operator-configured model endpoints cannot be selected through an agent manifest; the model receives no tools or wallet. See `HALO_PUBLIC_MODEL_RUNTIME.md` for exact releases and acceptance evidence.

Deployment configuration

Mainnet: chain 4663, public RPC <https://rpc.mainnet.chain.robinhood.com>, explorer <https://robinhoodchain.blockscout.com>.

Testnet: chain 46630, public RPC <https://rpc.testnet.chain.robinhood.com>, explorer <https://explorer.testnet.chain.robinhood.com>.

Public RPCs are for initial development and are rate limited. Supply independent production/archival RPC endpoints through configuration. Root HALO, its reference market, WETH and all deployed protocol addresses must be verified before funding. The SDK rejects unsupported public chain IDs.

Official source: <https://docs.robinhood.com/chain/connecting/>

Operating economics

Compute and gas are real expenses. A worker simulates the action, estimates fees, applies its own gas-cost ceiling and refuses work whose committed reward does not cover its compute budget and required margin. A local-only test override permits explicitly subsidized test work. It is rejected for testnet/mainnet configuration.

A local fee-funded cycle now passes: earned parent/base receipts convert through bounded routes into the operating token and cover subsequent proof-authorized work. `AgentFeeTreasury` isolates those receipts from trading capital; `HALO_FEE_SETTLEMENT.md` specifies the guards and keeper workflow. Sustained self-funding on real WETH markets and hosted compute remains an acceptance gate. Market capitalization, deposits and unconverted balances cannot be labeled operating income. Use measured chain-specific costs before setting the irreversible work reward.

03 / Implementation specification

Updated 13 September 2026. This supersedes earlier Solana-first implementation recommendations. It records the accepted product target, the current code boundaries and the remaining completion gates.

Production clarification and current delivery

The founder's PC is optional; production targets 100 active agents and hundreds of users on independently hosted cloud resources. This supersedes the initial 20-agent capacity gate. Shared model serving and isolated persistent workspaces avoid a dedicated GPU per agent. See [production hosting](#), [account setup](#) and [measured capacity](#).

The portable CPU operator/prover image and runtime Compose component are implemented and locally verified. A 100-proof benchmark is recorded separately from the still-required 100-agent cloud test. The public Operations API and website tab expose sanitized runtime outcomes. Headed desktop capture is implemented; general model-directed computer control and authenticated social workflows remain incomplete. No paid cloud lease, public testnet or mainnet deployment is claimed.

1. Product and network

HALO is a permissionless agent-token launch platform on Robinhood Chain, the Ethereum L2. All public protocol deployments, markets, wallet transactions and operating payments target that chain. Mainnet is 4663; testnet is 46630; local verification is 31337. Solana and cross-chain bridges are deferred.

The root HALO token's launch, allocation and initial market are outside this engineering scope. The root token address, compatible WETH address, reference market and operating capital are release inputs. A user must be able to connect a wallet, create and fund an agent, activate it, and see it launch and manage multiple child tokens autonomously. No approval is required for each permitted action after activation.

2. Launchpad and pairing decision

Meteora DBC supports permissionless SPL quote tokens and remains a future Solana option. PONS v2 requires quote-asset approval. The exact permissionless arbitrary-parent guarantees were not established for Pump.fun, Raydium LaunchLab, Long or Stonk. HALO therefore implements its own EVM curve; it does not depend on a script bypassing another protocol's allowlist.

FRED quotes in HALO. DOG and subsequent children quote in FRED. Spending existing FRED on DOG is not itself a new FRED purchase; upstream buying occurs only in routes that first acquire the parent assets. Outside markets cannot be forced to take those routes. Token ownership provides market exposure, not a guaranteed basket return or initial holder dividend.

References: <https://docs.meteora.ag/core-products/dbc/token-2022-support> ; <https://docs.ponsfamily.com/v2> ; https://github.com/pump-fun/pump-public-docs/blob/main/docs/instructions/COIN_CREATION.md

3. Contract graph

AgentRegistry creates immutable AgentVault instances and records public identity/manifests. CurveFactory accepts root HALO and factory-created tokens as quotes. HaloToken creates the fixed supply. HaloCurve owns inventory and backing. FeeSplitter fixes recipients and allocates actual deposited fees. AgentFeeTreasury separates operations receipts from trading principal; FeeSettlementRouter uses fixed parent/HALO/operating-token routes. RootReferenceMarket optionally bootstraps the externally supplied reference pool and observation hook. V4GraduationAdapter, HaloPoolHook and LockedLiquidityVault create and permanently hold a full-range Uniswap v4 position. EzklDecisionVerifier binds the actual generated verifier to the vault's authoritative inputs.

Active instances have no owner pause, treasury rescue, upgradeable implementation or mutable execution allowlist. New versions require new instances. Before activation, only the creator may withdraw funds; after activation, supported execution paths and immutable limits govern the vault. An operator gas wallet does not own the agent treasury.

4. Curve and graduation

Supply is one billion 18-decimal tokens, with $C = 800$ million on the curve and $L = 200$ million for liquidity. There is no later mint, transfer tax, blacklist or freeze function. The creator fixes quote target R at creation. The original virtual-reserve formula is $\text{virtualBase} = C^2 / (C - L)$, $\text{virtualQuote} = R * (\text{virtualBase} - C) / C$.

The implemented exact rational form is $\text{reserves}(s) = \text{ceil}(R*s/(4*C - 3*s))$; its inverse is $\text{floor}(4*C*r/(R + 3*r))$. Use conservative integer arithmetic, exact-input trades, minimum outputs, deadlines and explicit fees. Final purchases pull only the used quote amount; unused input stays with the payer. Virtual reserves are never withdrawable. Fees remain separate from backing.

At sellout, attempt same-pair Uniswap v4 graduation inside a gas-bounded subcall. Retain protected reserves if migration fails and allow anyone to retry. The hook restricts initialization to the migration adapter, and the liquidity vault has no principal-withdrawal or position-transfer interface. Quote-side earned fees enter the splitter. Accounted base fees can convert through the same pool under immutable price/depth bounds; seed residuals and principal remain excluded. Malicious initialization, repeated migration, integer rounding and price continuity belong in the invariant tests.

Use the official Robinhood v4 deployment registry and verify bytecode before release:

<https://developers.uniswap.org/docs/protocols/v4/deployments> . The latest local fork verified PoolManager 0x8366a39cc670b4001a1121b8f6a443a643e40951 and the new base-fee conversion at block 62029243; this is not a HALO mainnet deployment.

5. Fees and operating funds

Trading fees range from 25 to 200 basis points, default 100. Operations receive at least 50%; HALO receives 10-30%; the creator receives the remainder. Defaults are 60/20/20. Children inherit the committed policy. There are no initial token-holder distributions.

Only fees actually earned by HALO's curve or liquidity position count as trading revenue. Competing pools and third-party liquidity are outside that entitlement. Accounted parent/base receipts now convert through fixed routes with 30-minute observations, depth bounds and pre/post price checks. Direct treasury donations are excluded; splitter deposits still require payer attribution when reporting organic trading revenue. Unavailable or uneconomic routes leave fees pending. A local fee-funded proof-authorized launch and an independent settlement-worker transaction passed. Sustained profitability and production deployment remain unverified.

HALO_FEE_SETTLEMENT.md specifies the implementation, parameters and limitations.

Require the configured 30-day work reserve and trading inventory before activation. Below the seven-day reserve threshold, new launches and purchases are blocked; sales/hold and replenishment remain possible under their rules. Work rewards, gas, compute cost, deposits, realized trading results and unrealized estimates must remain separate accounting categories. The worker refuses work whose committed reward cannot cover its estimated costs unless explicitly running the local-only subsidized test mode.

At the default 1% curve fee and 60% operations allocation, eligible curve volume contributes 0.6% before conversion costs. Break-even volume = operating cost / (trading fee * actual attributable share * operations allocation). Graduated economics use actual collected LP fees. The \$1,500-5,000 monthly planning range is not a provider quote, engineering budget, audit budget or liquidity allocation.

6. Decision authorization

The current small public ONNX core is a fixed, untrained authorization graph. The real EZKL proof is integrated; the EVM adapter pins verifier bytecode and reconstructs 75 public instances from the commitment, ten Boolean policy facts, echoed commitment and authorization result. False facts, altered action bindings, corrupted proofs, wrong beneficiaries, wrong chains and replayed nonces cannot authorize valid spending in the tested implementation.

The action binds chain, agent, nonce, expiry, policy, core, market snapshot, evidence, amount, output limit and beneficiary. The vault independently checks actual funds and effects. Default limits are one launch per day, 10% maximum position, 10% daily trading debits and a 15-minute interval; bounded creator configuration is frozen at activation.

Current proof-authorized actions are child launch, purchase, sale and hold. New instances route trades to the curve until graduation, then to the immutable AgentTradeRouter and the child's official parent-quoted Uniswap v4 pool. Actual quote simulation, market-phase binding, TWAP/depth limits, exact input/output checks and proportional cost accounting passed real-proof and operator tests. Mechanical fee claims and operating-reserve conversion use fixed destinations; permissionless keepers receive at most 1% of converted proceeds, capped at the committed work reward. Realized-profit recycling remains. No arbitrary model-selected wallet transfer, unrestricted approval or generic contract call is allowed. See `HALO_GRADUATED_AGENT_TRADING.md`.

Recorded discretionary-trade quotes provide a minimum-output envelope and expire after 15 minutes; that snapshot alone is not a manipulation-resistant oracle. New curves separately integrate actual marginal prices for fee settlement. Graduated discretionary trades now also enforce the HALO hook's 30-minute tick history, depth and pre/post price bounds. Extending comparable controls to pre-graduation discretionary actions remains work. The public activity and stronger model-authorship requirements are specified separately in `HALO_PUBLIC_ACTIVITY_AND_SOCIAL.md`. The present proof does not prove that the larger LLM chose the action without human involvement.

7. Runtime and recoverability

The working worker reconstructs state from RPC and public artifacts, collects source snapshots, runs a typed proposal module, pins evidence, builds a canonical intent, proves, simulates, submits and verifies payment. Multiple children retain one parent identity. It can recover prior narratives from on-chain child evidence after losing its local files.

Manifests use canonical JSON and an on-chain Keccak commitment. New publications use real IPFS raw-block CIDs. Action evidence uses SHA-256 raw-block commitments, allowing the CID to be reconstructed directly from the event, including hold actions. Replication counts require distinct IPFS peer IDs, a reported pin and successful retrieval of matching bytes. This is an availability observation, not a perpetual guarantee or proof of independent corporate ownership.

Python proposal/proof processes receive only public files and an allowlisted environment. HTTP fetches enforce public address rules, bounded size, deadlines and pinned DNS answers. Committed public-model mode now runs Qwen3.5-4B Q8_0 on the local RTX 5090 through pinned llama.cpp; two real model-generated launches and a daily-limit hold passed the website-to-chain path. The model receives no tools or wallet. Prompt/schema/runtime commitments and public transcripts support inspection, not exclusive-authorship proof. Hosted Hermes/vLLM, independent model replicas and portfolio evaluation remain. See HALO_PUBLIC_MODEL_RUNTIME.md. The owner's local training is outside scope.

8. Website, API and public records

Retain the existing React/TypeScript/Tailwind/Vinext frontend. The founder's latest orange/violet palette, original inward-pointed ring, engraved portraits and selective glass treatment replace the earlier green direction; HALO_DESIGN_SYSTEM.md defines the visual system. Required routes are landing, explore, agent profile, creation, token detail, portfolio and transparency. Current local flows use actual contracts and data, not fake balances or confirmation timers. Trades stay on the curve interface until graduation; external trading destinations apply to supported public networks.

Creation separates draft, manifest publication, contract creation, inventory funding, reserve funding and irreversible activation. Resume from on-chain balances and activation state before asking for another transfer. Handle wallet rejection, wrong network, insufficient funds, stale quote, failed receipt and service unavailability. WalletConnect and remaining edge-case coverage are still required.

The read API uses Fastify/Zod and viem with direct chain reads. PostgreSQL 17/Drizzle now holds exact-nonce jobs, expiring fenced leases, attempt history and an ordered transactional outbox; runtime/cli.mjs uses the scheduler in execution mode and publishes receipt artifacts through the outbox. Canonical block/event projections, accounting, completed-job reorganization correction, queued broadcast recovery and website queue visibility remain. Twelve real-database scenarios and four local chain/IPFS recovery scenarios passed. The native Windows restart failure remains recorded. A separate actual Linux run passed all twelve queue scenarios and a forced restart of the same PostgreSQL 17.11 container/volume, preserving jobs and pending publications with durability settings enabled. Independent operators can use their own databases, schedulers and RPCs. The website/API cannot be execution authorities.

The Live tab streams signed operator progress and public browser frames, independently checking completion receipts, frame signatures and image hashes. Publish theses, source evidence, wallet/holdings, immutable configuration, cost accounting and execution receipts. The founder confirmed fomo.family and selected visual browser interaction for X/FOMO. The isolated worker, constrained egress proxy, publication reconciliation and durable signed-frame publisher are implemented with local boundary and transport tests. The actual Linux worker now opens FOMO with Chromium sandboxing enabled; three signed reports and two images reached Nova's website viewer. Eleven container checks and real publisher lock exclusion passed. X returned HTTP 403 and is reported unavailable. Startup errors now produce signed image-free failure reports. A subsequent real queued launch job now opens FOMO account setup through the Linux runner, delivers five signed reports and one public image, and remains pending without fabricating an account. The profile lock, lease renewal, interruption cleanup and standalone X-error delivery passed. Continuous native model workers now use the Linux PostgreSQL scheduler: a fresh Lyra/ARTEMIS launch triggered both actual social browser jobs, with five pipeline checks and seven signed reports. Eighteen persistence scenarios passed. Account signup, authenticated posting, portable model/prover hosting and independent deployment remain unfinished. The viewer distinguishes the local worker from earlier Codex development captures. Social availability must never control vault execution.

9. Hosting and completion gates

Use shared inference/proving capacity on Akash and portable CPU operators across at least two independent providers. Operators pay their hosting bills from WETH work payments. Publish pinned images and public recovery material on three independently operated IPFS copies. A lease can end; no single lease or HALO billing account may be an essential execution dependency.

The current local lifecycle, real proof, browser flows, IPFS storage, replacement operator, live timeline, bounded fee-funded work cycle and graduated agent trades are verified within their documented test scopes. Complete realized-profit recycling, continuous observation maintenance, stronger pre-graduation strategy-market and model-authorship evidence, portable model serving, complete durable accounting and recovery, authenticated social publication and independent public hosting next.

Then benchmark 20 active agents within 15-minute cycles and recover a replacement within 30 minutes. Run seven days on public testnet, with duplicate work, abandoned jobs, RPC failure, reserve exhaustion/replenishment, unavailable models/content and operator replacement. Commission independent economic, contract and proof review. Only after those gates and the supplied root HALO/reference market/operating funds are ready should mainnet release be considered complete.

Finished means a new user can activate through the site, the agent launches and manages children, earns and distributes actual fees, funds later independent work, graduates liquidity and exposes verifiable outcomes. A replacement operator can continue after HALO services and the original host disappear. The current implementation has not yet satisfied that whole definition.

Robinhood governance remains external: <https://docs.robinhood.com/chain/governance/>

04 / Agent framework

Updated 13 September 2026. Runtime target: Robinhood Chain, the Ethereum L2. The owner's RTX 5090 training work remains outside implementation scope.

Execution cycle

1. Read the chain, agent identity, nonce, policy, treasury and confirmed creation record.
2. Recover the manifest and every prior child narrative from content-addressed public artifacts.
3. Retrieve bounded public source data and pin the complete source snapshots on three distinct peers.
4. Run the public Python/Pydantic rules baseline, a committed public Qwen model release, or a strict custom proposal endpoint.
5. Validate sources, identity and funding/allocation limits. For a trade, recover or confirm a contract quote observation before publishing the complete proposal evidence.
6. Ask the vault for the exact action commitment and authoritative policy facts. Generate a fresh EZKL proof using the pinned public proving release.
7. Recheck the observed block, simulate execution, estimate gas and enforce the operator's economics.
8. Submit from the operator's own gas wallet. Verify the receipt, reward beneficiary and nonce; publish the outcome.

The contract interval defaults to 15 minutes. Local scripts support on-demand cycles and Nova's continuous model watcher. A PostgreSQL leased-job scheduler and receipt outbox are implemented, with Linux same-volume crash/restart acceptance now passed; independently deployed continuous workers and complete publication integration remain required. The contract enforces interval and replay rules even when operators compete.

Narrative modules

The working baseline selects unused public sources, scores topic overlap with the agent description and generates a clearly fictional community concept. It is deterministic engineering logic, not a trained model or demand forecast. Default research sources currently include NASA's public article feed and the Hermes project's public GitHub release feed. These are initial source adapters, not a finished memecoin strategy.

Source IDs are derived from public URLs. Prior source IDs are recovered from every child token's on-chain evidence commitment, so a replacement operator does not need the original operator's private memory. Missing or unverifiable history stops that worker from launching another narrative rather than silently forgetting earlier launches.

The custom API receives public interests, evidence, prior source IDs, launch eligibility and observed portfolio balances, cost basis, market phases and limits. It returns `halo.proposal.v1` for launch, hold, buy or sell. Trades require an owned child and positive integer input amount; models cannot choose destinations, routes or minimum outputs. The operator records or recovers the contract observation, proves the action and accounts for both observation and execution gas. A malformed response or unavailable endpoint does not silently select another model. The original public narrative baseline still only launches or holds; a deployed portfolio model and evaluations remain. See `HALO_GRADUATED_AGENT_TRADING.md`.

Proof and spending boundary

The public small core has 42 inputs: 32 action-commitment bytes and 10 Boolean policy facts. It echoes the commitment and authorizes only when every fact is true. The EVM adapter pins the generated verifier's runtime code hash and constructs its public inputs itself. A valid proof of a denied decision cannot authorize spending.

The action commitment binds chain, vault, immutable policy/core, full action, nonce, expiry, evidence, operator beneficiary and accounted treasury state. Live balances are checked as inequalities so unsolicited donations do not invalidate otherwise valid work. Trades use an immutable quote snapshot at least one block old, a 15-minute expiry and the committed slippage floor. Graduated trades also use 30-minute pool observations, depth and pre/post price checks. Pre-graduation discretionary snapshots alone remain insufficient as a manipulation-resistant oracle.

This proves admissibility under the current graph. It does not prove exclusive model authorship. A stronger model-output-to-action proof or measured-runtime attestation is needed for the new requirement that outsiders cannot directly choose the agent's decisions. See `HALO_PUBLIC_ACTIVITY_AND_SOCIAL.md`.

Models, training and hosting

The selected hosted stack is Hermes with configurable providers, vLLM and Qwen3.5-4B. Actual native Windows inference now uses pinned Qwen3.5-4B Q8_0 and llama.cpp b10809 on the RTX 5090. A website-created agent launched two children through different local operators, then held at its daily limit. The model, prompt, schema and adapter are committed; transcripts and source bundles are public IPFS artifacts.

`HALO_PUBLIC_MODEL_RUNTIME.md` records exact versions, timings and the remaining hosted/reproducibility boundaries.

Provide a model import/evaluation interface before accepting trained replacements. Export small decision models to ONNX, fix preprocessing and supported operators, quantize deliberately and evaluate on held-out temporal data. Package LoRA adapters with exact base-model and tokenizer hashes. Report execution cost, calibration, failures and out-of-sample results; a successful training run is not proof of profitability. The Qwen baseline now uses the owner's GPU for inference; no local training was performed.

Akash is selected for portable shared GPU inference and independent CPU operators. Each operator pays its own hosting bills from earned work payments. At least two independent providers and three independently hosted IPFS copies are required for the production design. The three current Kubo peers are distinct processes on one local machine and do not satisfy that deployment requirement.

Hermes provider documentation: <https://hermes-agent.nousresearch.com/docs/user-guide/configuration>

EZKL: <https://docs.ezkl.xyz/>

Akash deployments: <https://akash.network/docs/learn/core-concepts/deployments/>

05 / Public activity and social channels

Updated 13 September 2026. Added to the accepted product scope. This document distinguishes implemented behavior from work and external dependencies that remain.

Product requirement

Every agent is a persistent coin deployer on Robinhood Chain, the Ethereum L2. It can develop multiple narratives and launch multiple tokens. People should be able to inspect its thesis, treasury, holdings, costs, proof and transactions, and watch its current work. X and FOMO accounts are requested distribution channels.

Implemented live view

The agent profile now has a Live tab. The worker emits: read chain state, retrieve research, publish proposal, generate decision proof, simulate and check costs, submit, then confirm. Skipped work and errors have explicit outcomes. The stream uses server-sent events and reconnects without wallet access.

Records include the chain, registry, agent, execution nonce, operator, run ID, sequence number, previous record hash, timestamp and a bounded public summary. The operator signs the canonical record. The browser verifies its hash and signature. For a completion report, the browser also retrieves the transaction receipt and checks the agent address, execution nonce and reward beneficiary. A signed report alone is never labeled a verified receipt.

The local implementation has completed a seven-step cycle with a real EZKL proof and real local contract execution. Tampered reports, conflicting history, cross-chain records and a different operator claiming the receipt are rejected. The file journal survives a service restart. Production still requires the durable PostgreSQL projection, retention/rate controls, independent stream hosts and reorganization handling throughout the displayed history.

What the evidence proves

Evidence	What can be checked	What it does not establish
Contract bytecode and activated policy	No owner pause, rescue, upgrade or policy-change method in these instances; enforced spending and action limits	Continued compute availability or immunity from chain governance
Current EZKL decision proof	The pinned small ONNX authorization graph ran on the commitment and authoritative Boolean policy facts accepted by the vault	That a larger LLM ran, that its thesis is true, or that the proposer was uninfluenced by a person
Content-addressed evidence	Retrieved bytes match the public content commitment	Truth of the source or indefinite future availability
Operator-signed timeline	A particular operator signed these statements; alterations can be detected	A person could not have instructed that operator or fabricated an intermediate step
Confirmed receipt	The chain accepted that exact action under the vault's checks and paid the bound beneficiary	Guaranteed profitability or full Ethereum settlement finality from a small confirmation count
Desktop livestream	Viewers can observe a screen	Absence of hidden control, edited footage or unrecorded processes

The present public rules core allows any operator to propose an action that satisfies the immutable envelope. It does not prove exclusive model authorship of that proposal. This is a material distinction for the new request for proof that only the agent can influence decisions.

Stronger public execution mode

Add a separate, versioned mode before claiming model-authored autonomy. Commit the complete model weights, tokenizer, prompt templates, tools, deterministic input construction, fallback rules and runtime image at activation. Bind the decision and complete input snapshot to the nonce and current chain. Benchmark a proof of the actual small decision policy, including how its output determines the action, rather than a proof that merely confirms admissibility facts.

Where a full LLM proof is too expensive, evaluate an attested confidential-compute worker with measured boot, a pinned container and a key usable only under the approved measurement. Publish the attestation, trust roots, freshness challenge, public input commitments and output receipt. This introduces hardware-vendor and attestation-service trust; it is not mathematically equivalent to proving the whole computation. Custom API mode must continue to disclose that an endpoint owner can change its responses.

A visual browser is now an explicit product requirement. The Live tab includes a read-only browser viewer beside the signed timeline. A separate relay validates PNG/JPEG image digests, operator signatures, sequence history and deployment identity, then serves frames through SSE. Viewers can pause, resume the latest screen and expand the frame. Private/account-required states carry no image. A Codex-controlled capture of FOMO's public homepage was published and verified in the local Cedar profile; it is explicitly labelled a development capture, not an autonomous production run.

Ten relay checks passed, including signature/image tampering, dimensions, private-state suppression, sequence forks, idempotency and restart recovery. The Linux browser worker now includes continuous capture, an

isolated-network proxy, a pinned image definition and a separate trusted signed-frame forwarder. The forwarder persists pending signed records before sending, reconciles lost acknowledgements and emits image-free gaps for expired captures. Eleven deterministic driver/capture checks, ten forwarder checks, nine egress checks and five actual loopback CLI checks also passed. The egress checks use injected DNS/TCP, and the driver uses a DOM fixture; these do not establish that the Linux container or authenticated social sites work. A subsequent actual Linux build and ten container checks verified restricted privileges, resource limits, root immutability, controlled network routes, profile separation, user-namespace creation and cleanup. A later real worker run established sandboxed Chromium startup and public FOMO observation, with three signed reports and two images verified in Nova's website. Real journal locking excludes a second publisher, and a forced startup failure reaches the viewer without images. Thirteen driver fixtures now include HTTP-error and unfinished-page rejection. Authenticated social interaction and actual accounts remain acceptance work. Social credentials belong only in the isolated private profile and must never appear in streamed frames; treasury keys remain outside the browser container. Recording must not become a dependency of vault execution.

X integration

The official API supports creating posts at POST <https://api.x.com/2/tweets> using an authorized user context. The reviewed documentation did not establish a public endpoint for creating new X accounts. X's automation rules require transparency and restrict duplicative automated accounts, unsolicited interactions and non-API automation.

The founder explicitly selected browser-based interaction for the visual experience. Implement capability-aware states: account required, authorization required, connected, credentials expired, rate limited and disconnected. The browser driver uses named visible controls and an isolated persistent profile; it does not have treasury authority. Generate profile text and links from the immutable identity. OAuth/API posting remains an alternative adapter. Neither route makes an unsupported promise that new accounts can always be registered without identity checks.

Posts should contain a unique thesis summary, agent wallet address, parent/child relationship, relevant transaction link and a link to the public evidence. Use a durable publication outbox keyed by chain, agent, action nonce and destination. Treat an uncertain API response as an unknown outcome to reconcile; do not blindly post again. Keep social credentials outside model/prover containers. Platform account suspension or credential revocation must not pause on-chain execution.

The actual isolated worker received HTTP 403 from X; it now records site-unavailable and emits no screenshot. This environment has not demonstrated an accessible X session. No X accounts have been created or connected and no posts have been sent in this implementation session. OAuth credentials and authorized accounts remain external setup dependencies.

FOMO integration

The founder confirmed <https://fomo.family/>. The public homepage and login dialog were inspected in the browser. The observed login offers Apple and Google. No identity was entered and no account was created. An official account-creation or thesis-posting API has not been established; the selected integration is the browser driver.

The driver opens the public site and reports an account-required state until an authenticated identity is available. Publication requires verified bindings for the actual signed-in profile and composer. It journals drafted/submitting/posted/uncertain states and binds the exact text and destination to the job. An uncertain submission now triggers a read-only profile check, requiring a unique matching thesis, the configured author and a public post link. Unresolved or ambiguous outcomes remain uncertain without an automatic duplicate post. The composer, Robinhood Chain support and contract-vault linkage still need real-account verification. Treasury funds must not be exported to a separate FOMO wallet to imitate compatibility.

HALO's own thesis feed and wallet pages remain authoritative. FOMO can mirror public summaries and link back to the agent vault. The code is being implemented under the user's requested browser approach; account availability and platform behavior remain observable external dependencies.

Actual queued browser execution

The durable social intent now runs the real Linux container through a separate runner and social CLI. Recovering Nova's first confirmed launch produced an X intent and a FOMO intent. The FOMO job opened the actual account-setup flow, emitted five signed reports and one public image, renewed its PostgreSQL lease during execution and stayed queued with needs-account. Its session was 217098fc-6f55-4f6e-8596-7d96a27e06f1. Nova's website verified the image hash and signature and suppressed the private account screen. No account or social post was created.

The runner uses asynchronous subprocesses and a real Linux flock for each chain/registry/agent/platform profile. A second invocation could not acquire the active profile. Cancelling an authorized run removed its browser, proxy and networks, retained the private profile and released the lock. A replacement holder first cleans any containers left by its predecessor. The publisher and signing key stay outside Chromium; inherited database credentials are excluded from subprocess environments. Receipt and lease checks run immediately before authorizing container startup; they are not an atomic transaction with a later external Publish click.

The rebuilt image is sha256:437700fd2cb850cbf86e1629a48eb37097651baf63168651ed210c1eea78b650 (local image ID). Eleven real container boundary checks passed on this image. Seven receipt/outbox regression scenarios also passed. The standalone social CLI exercised the X job: actual HTTP 403 produced two signed image-free reports and a deferred database state. Selected evidence is in [halo-protocol/test-results/linux-queued-browser/](#).

The continuous local model workers now execute through the same PostgreSQL scheduler used by the runtime CLI. Agent-scoped claims prevent one worker from consuming another agent's job. A host-key-pinned SSH connection links native Windows inference/proving to the accepted Linux database; the separate Linux social consumer picks up newly committed launch intents. Lyra's fresh ARTEMIS launch passed five end-to-end pipeline checks and generated seven actual signed X/FOMO browser reports. FOMO remained needs-account and X site-unavailable; no account or social post was created. Eighteen persistence scenarios passed, including scoped concurrency and persisted publication metadata. Independent hosting, account provisioning, authenticated publication, automatic recovery of incomplete frame delivery and public queue/thesis pages remain unfinished. All these services still share one physical PC.

Remaining implementation and acceptance

PostgreSQL leases and the transactional receipt outbox are now implemented and tested against local chain/IPFS evidence. The social worker is now connected through the separate Linux runner, with actual queued-browser acceptance. The existing in-memory signed-decision-step publisher has not yet moved into this outbox. Native Windows database crash recovery failed because PostgreSQL could not signal its checkpoint process; A subsequent independent Linux test volume passed all twelve queue scenarios and same-container crash recovery. The earlier Windows data was preserved; this does not yet complete continuous operator/publication integration. See [halo-protocol/services/persistence/README.md](#) for the precise implemented boundary and the retained failure evidence.

1. Complete durable, replicated event projections and reconnect/reorganization tests for the live feed.
2. Add source/thesis views, stable evidence links for every confirmed action and exportable receipts.
3. Move the connected model producer, accepted Linux worker and PostgreSQL outbox onto independent providers, and test with authorized accounts. The native model producer already uses the local Linux database. The exact operator procedure is in `halo-protocol/runtime/browser/README.md`.
4. Verify the authenticated X/FOMO profile and composer controls, identity linkage, publication reconciliation and account-required recovery. FOMO's identity is resolved; automatic signup has not been demonstrated.
5. Benchmark stronger action-determining proofs or measured-runtime attestation; document exactly what is verified.
6. Test social outages, expired credentials and unavailable streams while independent on-chain work continues.

Primary references

- Robinhood network details: <https://docs.robinhood.com/chain/connecting/>
- X create-post endpoint: <https://docs.x.com/x-api/posts/create-post>
- X automation rules: <https://help.x.com/en/rules-and-policies/x-automation>
- X developer guidance: <https://docs.x.com/developer-guidelines>
- FOMO product: <https://fomo.family/>
- FOMO terms: <https://fomo.family/terms>
- Kubo content and pin APIs: <https://docs.ipfs.tech/reference/kubo/rpc/>

06 / Evidence and completion gates

Updated 13 September 2026. Active implementation; no public testnet or production launch is claimed.

Production capacity and portable runtime checkpoint

The target is 100 active cloud-hosted agents with no required founder PC. The owner confirmed no existing cloud account. `HALO_PRODUCTION_HOSTING.md` and `halo-protocol/deploy/runtime/README.md` now document topology, provider shortlist, budget/account setup and the runnable CPU component. The image pins Node/Python bases and dependencies, verifies its source manifest at startup, runs as UID 1000, and passes six actual container checks. The checked image identity and source manifest are recorded in `halo-protocol/test-results/runtime-container.json`; no registry publication or paid deployment is claimed.

100 real CPU decision proofs completed in 117.856 seconds at concurrency two under a 2-CPU/3-GB limit. Median was 2.366 seconds, p95 2.422, throughput 50.91/minute. That benchmark used the earlier image with the same prover; the final accounting update was checked with two additional real proofs. This is not full-agent or cloud capacity evidence.

The public Operations projection, restricted reader and website tab are live against the local Linux PostgreSQL database. Six projection/API scenarios passed. The desktop viewer also received actual headed Linux FOMO frames; login was suppressed, public playback rendered, and eleven container checks passed. The continuous consumer now selects the desktop image. No FOMO/X account or post exists yet.

Action-cost accounting was corrected to verify and include the committed market-observation receipt, recover it from `calldata/events`, distinguish different gas payers and avoid false batch allocation. Eight boundary cases, six existing local actions and three persistent scheduler scenarios passed. Source changes apply to newly started workers; existing historical receipt artifacts are retained.

Current completion board and newest checkpoint

[Open the maintained green/yellow/red progress board](#) for the current product-wide status. This document retains detailed historical checkpoints; later verified work supersedes earlier pending items.

At **20:34 UTC**, AgentMail accepted the owner's code and confirmed verification. Lyra's previously restricted request then created its separate inbox. A repeat provisioning run reused both Nova and Lyra resources without increasing the two-inbox inventory. Six live checks verified provider access, distinct inboxes, deterministic identity metadata, private database mappings to activated vaults and existing launch evidence. Message reception and X/FOMO accounts remain unfinished. The owner code is no longer pending. Selected evidence: `halo-protocol/test-results/agent-inboxes-live.json`.

At 20:06 UTC, Lyra launched ARTEMIS from a real Qwen3.5-4B proposal through the PostgreSQL scheduler and a fresh EZKL proof. Transaction `0x8501ec55d646a8ab80706e6c8178f2aa6439e72616b671566e3ec53c35cd6928` confirmed at local block 676 and paid 0.01 test WETH. The cycle took 4.687 seconds, including 2.073 seconds of inference and 0.470 seconds of proving. The same committed result created X/FOMO intents; actual Linux browser runs delivered seven signed reports. Five pipeline acceptance checks and eighteen persistence scenarios passed. FOMO still needs an account; X reports unavailable. No social publication occurred.

The outbox now preserves receipt-publication metadata on acknowledgement. Three older local receipts whose metadata was omitted were reverified against their canonical transactions and re-pinned without a new transaction or social post; recovery is explicitly recorded. Selected evidence is in

halo-protocol/test-results/model-social-pipeline.json and receipt-metadata-recovery.json. Native model/prover processes now feed the Linux queue continuously, but independent hosting and complete recovery/accounting remain unfinished.

Durable email identity checkpoint

Private provider/inbox tables and an independent provisioning CLI now exist. Eleven mailbox scenarios passed on Linux PostgreSQL, and the existing eighteen persistence/seven social-outbox scenarios passed under additive schema 3. A separate actual provider check recovered Nova's initial inbox and its activated-vault mapping. Lyra's first create request was access-restricted; the newer owner-verification checkpoint above resolved that restriction and confirmed its inbox. FOMO's actual signup dialog currently offers Google and Apple, without direct email input. See HALO_AGENT_IDENTITY.md for the required identity-provider step and exact limits.

Lyra subsequently launched LUNAR SCOUT at block 739 through transaction 0x2d2582ae646782eb54e643a8c34cd8e784d61c785c1ef42ef96187a4dfba4799. One repeated-source proposal was rejected before the later valid cycle succeeded. The successful cycle took 5.601 seconds and its new receipt publication metadata persisted without backfill. Mail setup did not stop chain work.

Latest identity update

The founder's supplied textured orange-gold ring is now the website logo. Its purple background and center were removed to transparent alpha. Header, footer, market hierarchy and browser icon use optimized PNG exports; the current downloadable logo pack and design notes match. The seven-route production build and TypeScript check passed. The in-app browser verified loaded assets, Home -> Explore -> HALO home, desktop and 390px phone layouts, and no relevant console errors. This update is available in the existing local preview.

AgentMail accepted the owner-authorized signup at 19:55 UTC and created halo-nova-532323de@agentmail.to. The organization key and response are encrypted with Windows DPAPI outside the website. Owner verification and a second inbox subsequently succeeded at 20:34 UTC; message reception and automated X/FOMO signup remain unfinished. No personal owner email, verification code or provider key was added to public website files. See HALO_EMAIL_PROVIDER_RESEARCH.md.

Target

HALO runs on Robinhood Chain, the Ethereum L2: mainnet chain ID 4663, testnet 46630, ETH gas. Local Anvil tests use 31337. The root HALO token and its initial reference market are supplied deployment inputs. Solana and bridges are deferred.

An agent is a persistent coin deployer. It can discover multiple narratives and launch multiple child tokens under one immutable policy. Agent tokens quote in HALO; their children quote in the agent token. Public activity, theses, wallet records and requested X/FOMO distribution are part of the product scope.

Working locally

- Custom fixed-supply curves, integer accounting, parent quotes, fee splitting, partial fills, migration retries and actual Uniswap v4 graduation into permanently held liquidity.
- Immutable activated vaults with repeated child launches, curve purchases/sales, spending limits, replay protection and independent-operator work payments. New instances now also execute proof-authorized purchases and sales through the official graduated Uniswap pool.

- A real pinned EZKL verifier is connected to authoritative vault inputs. Fresh proofs authorize actual launches, buys and sells. Altered bindings, denied decisions and stale quotes are rejected.
- A Fastify read API projects actual chain state. The website displays agents, tokens, holdings, fee balances, policy, operating runway and action receipts.
- Browser-tested creation, funding, activation, purchase, sale and creator-fee claim, including wallet rejection and restored drafts. The website-created Cedar agent then launched two different child tokens through two operator wallets.
- Python/Pydantic narrative proposals, bounded public-source ingestion, complete source snapshots and real IPFS pin/retrieval across three distinct local Kubo peers.
- A replacement operator recovered public state and launched a new narrative without HALO's API, the original operator's files or its original IPFS peer endpoint. This is a local fault-injection test, not evidence of independent cloud providers.
- A Live profile tab streams signed research/proposal/proving/submission steps. The browser verifies signatures and checks completed actions against RPC receipts. The latest tested cycle contains all seven phases.
- A strict custom proposal API adapter and committed public-model mode exist. Actual Qwen3.5-4B inference now runs on the RTX 5090 through pinned native llama.cpp and has driven two child launches and a hold through real proofs and the website. Hosted Hermes/vLLM remains unfinished.
- The Live tab now displays signed public browser frames with image-hash verification, pause/latest controls and explicit private/development states. Ten relay checks passed. The original Cedar capture came from Codex; Nova now receives actual Linux-worker FOMO observations. Account creation and authenticated social publication remain unverified.
- Browser reports now have a durable signed delivery journal and a real HTTP publisher. Lost acknowledgements and process restarts preserve the exact signed record; expired images become explicit gaps. The social driver reconciles an uncertain post by checking the configured author, exact thesis and unique public link without posting again. An isolated Linux image, internal network/proxy configuration and operator guide are supplied; the final image passes eleven actual Linux container checks and sandboxed FOMO observation with signed delivery. Authenticated accounts and posting still require real session tests.
- The website now uses the requested orange/violet palette, original engraved portraits and the founder's textured eight-point inward-facing ring logo. Desktop and mobile search, profiles, playback, wallet dialog and creation validation were inspected. Transparent PNG exports are supplied; earlier SVG marks are retained only as historical workspace assets.
- PostgreSQL 17.11/Drizzle now supplies nonce-keyed jobs, expiring fenced leases, attempt records and an ordered transactional outbox. The execution CLI uses this scheduler and publishes recovered receipts through IPFS. Real-database concurrency and local receipt recovery passed before the native restart drill; the original native database server remains stopped after the Windows failure; a separate Linux test server now passes concurrency and crash recovery. Queue status is not yet exposed on the website.

Verification evidence

The economic suite passed 19 scenarios, 116 successful local transactions and 12,000 seeded reference-accounting steps. The separate real-proof agent suite passed 7 scenarios and 38 successful local transactions after the fee-treasury changes. The latest Robinhood fork passed 15 local transactions at block 62029243, including collected base-fee conversion through the deployed Uniswap v4 PoolManager. Zero mainnet transactions were sent.

Ten settlement scenarios passed: isolated fee accounting, exact reconciliation of third-party claims, pending state on cold/stale/unsafe markets, principal preservation, curve and graduated conversion paths, actual fee-funded work and a paid independent worker. Three curve-oracle scenarios checked an independent piecewise time integral, a large atomic round trip and ring rollover. New instances use AgentFeeTreasury and FeeSettlementRouter; the original active agents remain immutable on their original chain.

The settlement preview on RPC 8546/API 8788 is connected through <http://localhost:5173/?preview=settlement>. Three agents converted actual locally earned HALO fees; Fred received 0.710610589529441219 test operating tokens after the keeper payment. Small child-fee balances remained uneconomic and were not traded. Browser collection rejection left balances unchanged, while confirmation moved 300 FRED from collectible receipts to pending conversion. Desktop 1280x900 and mobile 390x844 views had no horizontal overflow or relevant console errors; TypeScript and the seven-route build passed. The original RPC 8545/API 8787 and Cedar history remain intact.

Real action proofs measured approximately 0.4-0.5 seconds in the current small-core tests. Cedar's first operator launch took 2.163 seconds overall, used 3,206,731 gas and paid the configured work reward. Its replacement-operator launch recovered and executed in 3.039 seconds. These are small local observations, not production throughput commitments.

The first Cedar launch cost 0.003208355504270752 test ETH in gas and received 0.00005 test WETH. That original cycle was deliberately subsidized. The new settlement test converted 66 test HALO of curve-fee receipts into approximately 0.651397 net test operating tokens, then paid a real-EZKL child-launch reward of 0.01. The keeper and launch rewards both exceeded their measured local gas costs. This proves one local fee-funded cycle, not sustained profitability or paid cloud/model operation. The normal economic worker still refuses rewards below its configured cost budget.

Six transparency checks passed, including tamper rejection, duplicate idempotency, conflicting step rejection, wrong-chain rejection, receipt beneficiary verification and service restart. Runtime checks cover private/mapped IP rejection, DNS answer validation, redirects, byte limits, credential isolation, model-output validation and IPFS content integrity. TypeScript and the seven-route Vinext build passed; later UI changes are rebuilt before delivery.

The latest browser checks passed 11 deterministic driver/capture scenarios, 10 durable-forwarder scenarios, 10 relay scenarios, 9 egress scenarios and 5 CLI scenarios. CLI delivery used real loopback HTTP and disposable RPC signing. Egress checks used injected DNS and local TCP, not Docker network enforcement. The Windows host cannot spawn the nested Node test process, so the five CLI scenarios were exercised through separately launched execution-host commands. No external social post or financial transaction was sent by these tests. The relay was restarted with persisted frame history intact; the local chain was preserved.

Twelve PostgreSQL tests passed on version 17.11: 24 competing requests claimed 20 jobs without duplicates, expired workers lost acknowledgement rights, conflicting publications rolled back completion, and independent connection pools recovered stored state. Four scheduler tests recovered Cedar's actual nonce-0/nonce-1 receipts without another execution or block, rejected a stale nonce in the real operator, and published the receipts through the outbox to three local IPFS peers. Those tests sent zero new transactions and zero social posts.

The subsequent native database restart did not pass. WAL replay began, but PostgreSQL could not signal its checkpoint process (Operation not permitted) under the Windows execution host and shut down. Background-worker startup warnings had also occurred. The original data directory is preserved without resetting WAL or weakening durability. A Linux CI job now includes the same-database crash/restart drill, and the equivalent drill subsequently passed on an actual Ubuntu VM. The GitHub CI itself has not run. The existing local chain and website services were not restarted by this drill.

Machine-readable results live in halo-protocol/test-results/ and are ignored by Git. The directory also contains local service state and private IPFS peer identities. Publish only deliberately selected public evidence files, never the entire directory.

Graduated trading increment

Nine graduated-agent scenarios passed with 42 successful local executions. The real operator accepted typed proposal fixtures, recorded canonical observations, generated real EZKL proofs, bought and sold in the official v4 pool, and published receipts including both observation and execution gas. Manipulated markets, unsafe depth, cross-graduation snapshots, weakened output floors and stale observations were rejected. The existing economic, proof and settlement suites also passed after the change.

A separate preview is running on RPC 8547/API 8789 through <http://localhost:5173/explore?preview=trading> . It displays actual trade amounts and six completed actions for Graduation operator. This is a disposable scenario with controlled proposal fixtures, not hosted inference or a continuously executing agent. See `HALO_GRADUATED_AGENT_TRADING.md` for the architecture and exact verification scope.

The full graduated suite also passed on a local fork of Robinhood block 62043754, using the deployed, bytecode-verified Uniswap PoolManager: nine scenarios, 41 local executions, both real operator trade cycles and zero mainnet transactions. Desktop 1280x900 and mobile 390x844 browser checks verified the Activity amounts, payments, policy routing and graduated token page. A long mobile result wrapping mid-number was fixed with a compact display; the full value remains in Treasury. TypeScript and the final seven-route build passed.

Real model integration

Nova was created and activated through the website with a committed Qwen3.5-4B Q8_0 release. Actual inference on the RTX 5090 produced LUNAR_MIND and ASTRO_MIND; different local operator wallets launched them at blocks 77 and 80 of the preserved trading preview. A third real inference chose Hold at the two-launch daily limit and settled at block 83. Full cycles took 5.432, 5.445 and 3.185 seconds; inference took 2.793, 3.286 and 1.280 seconds. All three used fresh EZKL proofs, canonical receipts and public IPFS transcripts. No controlled proposal fixture was used.

The public model release is 4ece597fd8878b2a3c2d2030d6cccd2d7388d2eb9ae7776c9ceaa2da8df98b0e. Its native llama.cpp/CUDA archives, weights and extracted binaries were hash-checked before startup. A continuous local watcher now respects the 15-minute interval. Live model acceptance verifies distinct launch sources, both operators, all three receipt beneficiaries and retrieval of matching transcripts from three peer IDs. Adapter tests cover altered release files, endpoint separation, malformed/truncated responses, tool/recipient fields and credential isolation.

The website exposes Public AI model during creation and Inspect decision beside confirmed actions, with receipt/content verification, readable citations and public transcripts. Nova's live view shows all seven signed phases. Desktop and mobile checks identified and corrected cramped mobile evidence text. The model is an unevaluated public baseline and can misstate source implications. Its creator funded the test work reserve; these cycles do not establish recurring revenue, measured hosting costs or independent compute. See `HALO_PUBLIC_MODEL_RUNTIME.md`.

Linux acceptance

A separate Ubuntu 24.04.5 VM now runs through the Windows Hypervisor Platform already present on this computer. No Windows feature was installed, and the original HALO services stayed running. Node 22.23.2, Docker 29.1.3 and PostgreSQL 17.11 executed the real tests. The first emulated boot and its timing failure are retained as development evidence; the final VM uses hardware acceleration.

All twelve queue/outbox scenarios passed. A SIGKILL and restart used the same container ID and volume, then a different PostgreSQL process retained completed results and ordered pending publications. The recovered postmaster began at 15:54:31.912 UTC. Actual WAL replay and a completed recovery checkpoint appear in the

server log. `fsync`, `full_page_writes` and `synchronous_commit` remain on.

The pinned browser image built successfully. Ten checks against its real containers verified privileges, `seccomp`, read-only root, memory/CPU/process limits, internal-network isolation, restricted proxy access, user-namespace availability, mount separation and cleanup. Eleven driver and nine egress fixture checks also passed in Linux. No browser page was opened by these tests; no account or social post was created.

Evidence is retained in `test-results/linux-persistence/` and `test-results/linux-browser/`. The new Linux host is an acceptance environment on the same physical computer, not independent hosting. At this historical checkpoint the Nova watcher was separate; the later continuous-producer checkpoint connects native inference to the Linux PostgreSQL scheduler. Fully portable, independently hosted serving remains unfinished. See `HALO_LINUX_ACCEPTANCE.md` for the exact pins, scope and commands.

Actual browser-to-website execution

The actual isolated Linux worker now starts Chromium with its sandbox enabled, opens FOMO and streams signed screen reports to Nova's Live tab. The 19:17 UTC acceptance produced three reports and two 1280x720 JPEGs; their hashes and signatures were verified through the relay and in the website. The session was `c4572d4f-44ca-4124-bc0d-1545a7dfab3f`. This was an operator-triggered public observation, not an autonomous social publication or account creation.

The original startup failure was traced to the upstream `seccomp` profile's capability-conditioned `chroot` rule. With all container capabilities dropped, Chromium could not perform its filesystem sandbox step inside its new user namespace. HALO now permits that syscall while keeping the kernel's namespace-local capability check. Real probes verify that `chroot` fails outside the new user namespace and succeeds inside it. UID 1000, zero container capabilities, no-new-privileges, read-only root, private profile separation and controlled network routes remain enforced. The modified profile and its rationale are documented in `runtime/browser/container/THIRD_PARTY.md`.

A forced missing-browser failure on the final image emitted two signed reports, zero images and a failed/startup result; the publisher acknowledged the error without claiming task success. A competing publisher exited 75 under the real Linux flock. Thirteen driver/capture fixtures, eleven actual container checks, ten relay tests, ten forwarder tests and five real Linux CLI scenarios passed within their individual scopes. Windows nested CLI spawning still fails with `EPERM`; its Linux execution passed.

X returned HTTP 403 from this environment. The first observation incorrectly accepted the returned HTML shell; the driver now checks the HTTP response and waits for the expected visible application control. The corrected X run reports site-unavailable/403 with no image. Its access, account setup and authenticated interaction remain unfinished. No access controls were bypassed.

The final browser image is `sha256:4213a0dd722abf812805d840db8d7895c44249ca6cdcf41317f27a64faf1f70a` (local image ID, not a published registry digest). The trusted publisher runs outside the browser container and uses a host-key-verified SSH tunnel to guest loopback ports 8547/8795 for disposable RPC signing and relay delivery. No treasury key enters the browser. This tunnel is local acceptance infrastructure, not independent hosting. Selected public results and source hashes are in `halo-protocol/test-results/linux-browser-live/`; private browser profiles and signing journals are not exported.

The viewer distinguishes local Linux workers from Codex development captures, hides private/error images, stops the Live label on terminal errors and supports playback plus explicit fullscreen exit. Decision-triggered social jobs and PostgreSQL integration subsequently passed the later queued-producer checkpoint. Signed account linkage, actual X/FOMO signup/posting and wallet compatibility still require implementation and acceptance.

Confirmed-launch social outbox checkpoint

Scheduler completion now atomically stores an operator receipt and one social intent per platform for each confirmed launch. Two existing Nova launch receipts were recovered against the preserved local chain and generated exactly four X/FOMO intents. Replaying those intents created no duplicates; hold cycles generated no posts. The test submitted no new transactions.

Migration 0002 adds immutable prepared publication text and durable attempt outcomes. The social handler checks the canonical receipt, public evidence content hash and deployed token identity before preparing a thesis. Missing accounts remain pending. Once publication may have started, replacement workers retain the exact text and account and request read-only reconciliation. Lost private browser journals cannot authorize a fresh duplicate submission. Altered evidence, simulated orphaned receipts, changed accounts and mismatched reported post text are rejected.

Actual Ubuntu/PostgreSQL acceptance passed sixteen persistence scenarios and seven social-outbox scenarios. Fifteen deterministic browser-driver scenarios passed separately. The outbox test used real Nova receipts and public evidence but injected browser outcomes; it did not run Chromium, create an account or publish a post. The previous actual FOMO observation remains separate evidence. Source hashes for all 72 exported files were checked; selected results are in [halo-protocol/test-results/linux-social-outbox/](#).

This checkpoint used injected browser outcomes. The following increment exercises the queue through an actual rebuilt container and separately started social consumer.

Actual queued browser execution

The durable social intent now runs the real Linux container through a separate runner and social CLI. Recovering Nova's first confirmed launch produced an X intent and a FOMO intent. The FOMO job opened the actual account-setup flow, emitted five signed reports and one public image, renewed its PostgreSQL lease during execution and stayed queued with needs-account. Its session was 217098fc-6f55-4f6e-8596-7d96a27e06f1. Nova's website verified the image hash and signature and suppressed the private account screen. No account or social post was created.

The runner uses asynchronous subprocesses and a real Linux flock for each chain/registry/agent/platform profile. A second invocation could not acquire the active profile. Cancelling an authorized run removed its browser, proxy and networks, retained the private profile and released the lock. A replacement holder first cleans any containers left by its predecessor. The publisher and signing key stay outside Chromium; inherited database credentials are excluded from subprocess environments. Receipt and lease checks run immediately before authorizing container startup; they are not an atomic transaction with a later external Publish click.

The rebuilt image is sha256:437700fd2cb850cbf86e1629a48eb37097651baf63168651ed210c1eea78b650 (local image ID). Eleven real container boundary checks passed on this image. Seven receipt/outbox regression scenarios also passed. The standalone social CLI exercised the X job: actual HTTP 403 produced two signed image-free reports and a deferred database state. Selected evidence is in [halo-protocol/test-results/linux-queued-browser/](#).

The continuous local model workers now execute through the same PostgreSQL scheduler used by the runtime CLI. Agent-scoped claims prevent one worker from consuming another agent's job. A host-key-pinned SSH connection links native Windows inference/proving to the accepted Linux database; the separate Linux social consumer picks up newly committed launch intents. Lyra's fresh ARTEMIS launch passed five end-to-end pipeline checks and generated seven actual signed X/FOMO browser reports. FOMO remained needs-account and X site-unavailable; no account or social post was created. Eighteen persistence scenarios passed, including scoped concurrency and persisted publication metadata. Independent hosting, account provisioning, authenticated publication, automatic recovery of incomplete frame delivery and public queue/thesis pages remain unfinished. All

these services still share one physical PC.

Remaining product work

1. Realized-profit recycling, stronger pre-graduation strategy-market controls and continuous oracle maintenance. Graduated discretionary trades now pass the real contract and operator suites; the public baseline still needs a portfolio model. Base/parent fee conversion and a subsequent fee-funded proof action now pass locally; sustained economics across actual WETH markets and hosted compute remain to be demonstrated.
2. Hosted Hermes/vLLM deployment, versioned recoverable serving images, independent model mirrors and evaluated portfolio strategies. A pinned local Qwen service now generates real proposals through the full execution path; its baseline is not an evaluated trading strategy. Local training remains the owner's work.
3. Complete chain projections, reorganization handling, complete accounting, queued transaction-broadcast recovery, website queue visibility and durable live/social publication. Leased jobs and the transactional receipt outbox are implemented and locally tested; the whole persistence requirement is not finished.
4. WalletConnect, remaining error/recovery flows, public thesis pages, signed social linkage, X access and authenticated X/FOMO execution through the now-connected durable browser runner. FOMO is confirmed as fomo.family; authenticated accounts and composer behavior are not yet verified.
5. Stronger evidence of actual model authorship. The present proof enforces the admitted action envelope; it does not prove that the larger model selected the action without human input.
6. Akash deployments on at least two independent providers, independently hosted IPFS copies, pinned Linux images and observed CI execution.
7. Twenty-agent capacity tests, seven-day public testnet soak, recovery drills, independent contract/proof review and mainnet configuration/handover.

The current three IPFS peers share one Windows machine and run offline. The website and services are local. No production hosting, paid social account, real-money trade or cloud lease has been provisioned.

See HALO_IMPLEMENTATION_SPEC.md for the consolidated architecture and HALO_PUBLIC_ACTIVITY_AND_SOCIAL.md for the new live/social scope and its verification boundaries.

07 / Fee settlement and operating reserves

Implemented and verified locally on 13 September 2026. These contracts apply to new instances. Existing activated agents cannot be upgraded into them. The original Cedar deployment remains on its original local chain.

Funds and ownership

Each new AgentVault creates an immutable AgentFeeTreasury through the configured FeeSettlementRouter. The agent token and every child use that treasury as their operations recipient. Fee balances never share the agent's trading inventory. The treasury has no owner, withdrawal, rescue, pause or replacement-router method. Its only spending route converts accounted receipts into the configured operating token and pays the fixed agent plus a bounded permissionless keeper.

FeeSplitter tracks cumulative claims per recipient. The treasury recognizes the difference between that cumulative amount and its previous observation, so a third party can claim on its behalf without losing accounting or counting the same receipt twice. Direct token transfers into the treasury are excluded from pending fees. Deposits into a splitter can still be donations: financial reporting must attribute trade revenue using the actual curve/LP deposit events, rather than treating every splitter deposit as organic volume.

The root HALO token and operating token must be compatible ERC-20 assets. The production operating token is configured WETH. Disposable tests use an explicitly labelled fixed-supply test asset; they do not establish a real WETH deployment.

Routes and locked liquidity

HALO-denominated agent fees sell in the configured HALO/WETH reference pool. Child fees arrive in the parent agent token. Before its graduation, that token sells through its HALO curve; after graduation, the router selects the official agent/HALO pool recorded by that curve's immutable liquidity vault. HALO then sells in the reference pool. Callers cannot substitute arbitrary assets, venues or hooks in the treasury path.

LockedLiquidityVault collects quote fees into its immutable splitter. Collected base fees are tracked separately and can be converted through that same pool under SettlementGuard. It validates exact base input, actual quote output and post-trade price. The position's liquidity never decreases. Initial seed residuals are excluded from the convertible balance. Conversion costs reduce the distributed quote proceeds; only actual collected fees are attributed to HALO's position.

RootReferenceMarket is an optional bootstrap for a compatible v4 reference pool and HALO observation hook. A configured initializer sets the initial price once; ordinary v4 interfaces supply liquidity. This component does not mint or allocate HALO or promise permanent root-market liquidity. Public release configuration must verify the manager, token contracts, hook, reference pool, bytecode and actual liquidity before agent creation.

Market protection

Setting	Implemented rule	Failure behavior
Observation window	At least 1,800 seconds, at most 3,600 seconds from an actual retained boundary	Fees remain pending
Curve history	Integral of marginal quote/base price in Q128, updated at every buy/sell	Same-block price excursions have zero duration
v4 history	Integral of pool tick from the HALO hook	Mean tick rounded down conservatively
Observation storage	64 entries with at least 60 seconds between stored boundaries	Expired windows are rejected
Curve price deviation	Spot within 1% of the observed mean before and after conversion	Atomic revert
v4 price deviation	Within 100 ticks of the observed mean before and after conversion	Atomic revert
Trade size	At most 0.5% of real curve quote reserves or current v4 virtual input depth	Worker retries a smaller amount
Reference liquidity	Minimum WETH virtual depth fixed at router deployment	No conversion below the bound
Contract output floor	Observed mean output less actual swap fees and 2% tolerance per leg	Exact-input, minimum-output enforcement
Worker output floor	At least 99.5% of its freshly simulated gross output	Rejects excessive drift before mining

Curve marginal price is $4 \cdot R \cdot C / (4 \cdot C - 3 \cdot s)^2$. Each elapsed interval uses the price in force during that interval. Permissionless checkpoints update the stored history; they cannot invent past prices. A missing or old window may require checkpoints and real elapsed time before work becomes eligible. The local warmup helper advances only its disposable Anvil clock; it does not simulate elapsed public-network operation.

TWAP and depth bounds do not make thin markets immune to sustained manipulation. Current v4 liquidity is virtual depth at the active price, not the PoolManager's aggregate balance or proof of historical depth. Additional strategy, market-history and long-running economic evaluation remain necessary.

Uniswap v4 has no built-in price oracle, so the HALO hook is required for these routes:

<https://developers.uniswap.org/docs/protocols/v3/concepts/price-oracles> . Actual swap deltas are settled completely, following the manager's accounting interface:

<https://developers.uniswap.org/docs/protocols/v4/guides/unlock-callback-and-deltas> .

Independent operator workflow

runtime/settlement-worker.mjs reads the registry and immutable bindings directly from RPC. It does not need HALO's API to authorize a conversion. It rotates pages of child sources and limits each batch to sixteen tokens. A batch can collect LP fees, convert a bounded amount of base fees, reconcile splitter receipts and replenish the agent atomically.

Before signing, the worker simulates actual collection and conversion. It reduces oversized amounts, checks an explicit gas-cost budget and requires the minimum guaranteed keeper payment to cover that budget plus the configured margin. It never submits work solely because token balances look valuable. Insufficient history, unsafe markets and uneconomic work have distinct pending/skip states.

Keeper payment is the smaller of 1% of gross converted output and the agent's committed work reward. Net operating proceeds go only to the fixed agent. This mechanical work can replenish an exhausted agent; it does not require an action proof or use the agent's decision nonce. Discretionary launches and trades still require their committed proof and spending checks.

runtime/cli.mjs includes settlement rounds before decision work; settleFees defaults to true. Its production execution still requires the configured PostgreSQL service and operator-owned gas key. No production scheduler has been run. Permissionless checkpoint maintenance, uncertain-broadcast recovery, persistent settlement publication and economic operation across independent hosts remain acceptance work.

All transaction callers use gas headroom because a new observation/storage entry can become due between estimation and the next block. Test receipts report actual gas consumed, not the padded limit.

Website and local operation

Agent profiles show net reserve replenishment, keeper payments, accounted pending amounts and collectible splitter receipts in each token's actual decimals. Direct treasury transfers and uncollected LP fees are excluded from those fee rows. The current detail projection includes at most 100 child markets and discloses incomplete coverage. The wallet collection action handles the first sixteen displayed sources; the independent worker rotates through later pages.

Run scripts/dev-stack.mjs --settlement-preview to start a separate disposable RPC at 127.0.0.1:8546 and read API at 127.0.0.1:8788. The website selects it through <http://localhost:5173/?preview=settlement> in development only. Its per-tab selection preserves access to the original deployment through ?preview=legacy. Never restart the original chain to try this preview.

scripts/dev-settlement.mjs --warmup records real checkpoints, advances the isolated preview clock and runs a bounded settlement round. It rejects every RPC except that preview's loopback address and chain 31337. This is a test harness, not evidence of continuous production execution.

Evidence and limits

- The economic regression suite passed 19 scenarios and 116 local transactions, including conversion of collected v4 base fees without principal withdrawal.
- The separate settlement suite passed 10 scenarios. Actual fee proceeds replenished a vault, paid a keeper and covered a fresh real-EZKL child-launch reward. The independent worker also completed an atomic paid conversion and skipped work below its gas/margin budget.
- Three dedicated curve-oracle scenarios checked an independent piecewise integral, a large atomic buy/sell and history-ring rollover.
- The new v4 conversion path passed on a local fork of Robinhood block 62029243: fifteen local transactions, zero mainnet submissions. This fork test does not cover the entire proof/runtime system.
- The real-proof regression suite passed seven scenarios and 38 local transactions after the treasury change.
- In the persistent settlement preview, Fred received 0.710610589529441219 test operating tokens from fees. Browser rejection left its 300 FRED collectible; a confirmed collection moved that amount into pending conversion. Desktop and mobile layouts and console health were checked.

These results prove a local fee-funded cycle. They do not prove sustained profitability, autonomous model authorship, public testnet operation or an audited release. Root liquidity and activation reserves are externally supplied. Graduated discretionary agent trades now pass a separate contract and operator suite described in HALO_GRADUATED_AGENT_TRADING.md. Realized-trading-profit recycling, independent cloud/model operation, complete persistence integration, social accounts and the full public soak remain unfinished.

08 / Agent trading after graduation

Implementation record, 13 September 2026. Robinhood Chain is the target network. Verification described here used disposable local Anvil with real Uniswap v4 and EZKL contracts, test assets and controlled model-proposal fixtures. No public-chain transaction or hosted model deployment is claimed.

Lifecycle and authority

New AgentVault instances can buy and sell their own children both before and after graduation. The existing BuyChild and SellChild action types retain their nonce, interval, proof, beneficiary, position, daily-debit and operating-reserve checks. Graduation does not give an operator or creator additional authority.

FeeSettlementRouter creates an immutable AgentTradeRouter at deployment. The agent commits both addresses in its policy hash. The trade router resolves the child's official curve and locked liquidity vault from CurveFactory, checks the configured PoolManager and verifies the exact child/parent currencies. Models cannot select pools, hooks, spenders, recipients or arbitrary calls. Old activated instances remain unchanged and do not gain this capability.

Before graduation, the vault continues to use HaloCurve's exact-input buy/sell methods. After graduation, the vault approves only the input amount to AgentTradeRouter, receives the output itself and clears the approval. It checks both input and output balance changes. Purchases update daily debits and position cost; sales remove proportional cost and update realized profit/loss and accounted capital. The liquidity vault's permanently held principal is never removed by these trades.

Quotes, proofs and market checks

snapshotTrade(child, kind, amount) records a quote, nonce, block, timestamp and market phase. A snapshot from the curve cannot authorize a graduated trade. Observations must be from an earlier block, last at most 900 seconds and bind the exact asset, direction and amount. The action's minimum output cannot be weaker than the creator's committed slippage limit.

For graduated pools, AgentTradeRouter.quote executes the actual v4 swap inside a reverting subcall. A narrowly decoded QuoteOutput error returns the exact quote. All pool writes, oracle updates, fee growth, token transfers and swap logs from the simulated swap revert. Other errors propagate. A mined quote transaction therefore also leaves the pool unchanged.

The graduated route shares the fixed SettlementGuard bounds:

- At least 1,800 seconds of observed history; the selected observation window must not exceed 3,600 seconds.
- Pre-swap and post-swap ticks within 100 ticks of the time-weighted mean.
- Input at most 0.5% of the current active liquidity's virtual input depth.
- Output bounded by the observed price, actual direction-specific swap fees and the fixed 2% guard tolerance; the action's own output minimum also applies.
- Exact full-input settlement, expected PoolManager callbacks and no caller-selected output destination.

The proof's tenth public fact checks funds, snapshot and live market admissibility for a graduated trade. It is not a proof that an arbitrarily high minimum can be filled. The real swap independently enforces the actual minimum and atomically reverts the whole action if it cannot be filled. The unchanged EZKL graph proves the Boolean authorization envelope, not large-model reasoning or profitable prediction.

Curve snapshots remain slippage observations; stronger pre-graduation discretionary market controls are unfinished. Time-weighted price bounds do not establish immunity to sustained manipulation of thin markets.

Model and operator integration

The strict custom proposal interface now accepts `launch`, `hold`, `buy` and `sell`. A trade contains an owned child address, positive integer input amount, rationale and retrievable public source IDs. It cannot contain a recipient, route, minimum output or launch metadata. Buy input units are the parent token's raw units; sell input units are the child's raw units.

Custom providers receive the observed portfolio, balances, cost basis, market phase, immutable limits and daily debit. Public-source reuse is allowed for management of existing positions; launch proposals still cannot reuse an already-launched narrative source. The original public narrative baseline retains its launch/hold behavior; a deployed portfolio model and evaluation are still required.

The operator validates ownership, available balance, reserves and allocation limits before paying for an observation. It recovers a recent matching observation when possible; otherwise it simulates and budgets a new observation transaction, waits for at least two confirmations, then generates the real proof and simulates execution. Output floors come from the contract. Both observation gas and execution gas are counted against the work economics.

A dry run without an existing observation returns `observation-required` and sends no transaction. It cannot claim to have proved an executable action against nonexistent persistent state. A prepared observation does not guarantee the eventual action: market changes, competition or a failed proof can leave its gas cost unreimbursed. Full uncertain-broadcast recovery remains part of the persistence work.

Accepted receipts must come from the agent, match the commitment, action, nonce, evidence and beneficiary, and remain in the canonical block. Public evidence includes the portfolio observation, quote transaction, proposal and source snapshots. Public receipt records include input, output and both gas costs.

Website and local preview

Start the separate, disposable preview after compiling:

```
node test/graduated-agent.mjs --no-compile --preview
```

This runs the real acceptance scenario, then preserves its chain on RPC 8547 and exposes its public API on 8789. Open <http://localhost:5173/explore?preview=trading>. It does not restart the original preview on 8545 or the fee-settlement preview on 8546.

The profile displays real input/output amounts and operator payments in Activity. Policy & proof identifies whether the particular immutable agent supports the official graduated pool. The token page shows its parent pair, completed graduation and locked liquidity. Local results are explicitly identified as test results. The scenario contains six completed agent actions, including buys and sells after graduation. The provider is a controlled test fixture, not a continuously running LLM.

Evidence

`halo-protocol/test-results/graduated-agent.json` records nine passing scenarios and 42 successful local executions, including two complete real operator cycles. Checks cover market transition, insufficient history, quote rollback, callback/market rejection, impact limits, action tampering, actual output enforcement, manipulation invalidating a proof, exact accounting, locked LP principal, expired observations, zero-write dry runs and duplicate-cycle rejection.

The original economic suite also passed 19 scenarios and 116 local transactions, the existing real-proof suite passed seven scenarios and 38 transactions, and the fee-settlement suite passed ten scenarios and 45 executions after this change. Provider responses in the new operator test use a controlled transport fixture, research uses loopback HTTP, and content-addressed artifacts are held in memory. Independent model hosting, social accounts, public deployment, sustained economics and independent IPFS hosting remain separate acceptance requirements.

The full nine-scenario suite also passed on a read-only Robinhood Chain fork at block 62043754, with 41 successful local executions and zero mainnet transactions. It used the deployed PoolManager at `0x8366a39cc670b4001a1121b8f6a443a643e40951` and verified its bytecode hash before execution. This includes both actual operator trade cycles and real proofs, while retaining the disclosed proposal and storage fixtures. Evidence is in `halo-protocol/test-results/graduated-agent-fork.json`. Run `node test/graduated-agent.mjs --no-compile --robinhood-fork` to repeat against a fresh verified fork.

09 / Public model runtime

Verified locally on 13 September 2026. This is actual model inference connected to the website, proofs and disposable chain transactions. Public hosting and production acceptance remain incomplete.

Pinned implementation

The native Windows development service runs Qwen3.5-4B Q8_0 on the owner's RTX 5090 through llama.cpp. The selected Linux Hermes/vLLM stack remains a separate deployment task; the GPU backend stays native while the new Linux acceptance VM verifies database/container behavior. Training was not performed or required for this integration.

Component	Exact selection	Commitment
Model	bartowski/Qwen_Qwen3.5-4B-GGUF, Q8_0	Repository revision 4168f45a16a1290d65a4ec0fa312ae917a4c15d6
Runtime	llama.cpp v0.4.0, build b10809	Source commit 5266f24da75dc449bd56cbcd7addb9c8e4a6a73e
GPU backend	Windows x64, CUDA 13.3	Official runtime/DLL archive digests in assets.lock.json
Serving	Loopback port 8080, one slot	16,384-token context; FP16 KV cache; full GPU offload
Generation	Temperature 0.3, seed 42	1,024 output-token ceiling; reasoning disabled

Model SHA-256: 5c74c0ede371924357dff0cb6ba145bd67208b9b2389ded681adfff3f7608db7.

Nova's proposal-release SHA-256: 4ece597fd8878b2a3c2d2030d6cccd2d7388d2eb9ae7776c9ceaa2da8df98b0e.

The release binds weights, runtime source, prompt, output schema and adapter source. Startup verifies the downloaded assets and every extracted runtime file against the verified archives. The inference server has a local API key and no enabled tools, shell or wallet. Operator credentials are absent from public inference requests and transcripts.

Activation and execution

Creation now offers Rules baseline, Public AI model and Custom API. Public AI model commits proposalReleaseSha256 with public-weights reproducibility; its manifest cannot select a private endpoint. Existing activated rules/API manifests remain supported. An operator explicitly binds a committed release to its own loopback HTTP or public HTTPS inference backend. A missing release never silently falls back to another model.

The model receives public narrative interests, bounded source evidence, used-source history, portfolio balances and policy limits. It returns a launch, hold, buy or sell proposal. Action-specific JSON grammar prevents mixing launch and trade payloads. Independent Zod checks retain string limits and semantic validation. This separation was required because expanding long Unicode-string limits into llama.cpp grammar exceeded its grammar-complexity ceiling.

The operator still checks sources, launch eligibility, owned assets, allocation limits, output floors, nonce, proof and economics. It publishes the release, prompt/schema/adaptor bundle and complete parsed inference transcript to three verified IPFS peers. Evidence commits their CIDs. Full model weights currently reside in the local cache and at the pinned upstream repository; three independent full-weight copies remain a production hosting requirement.

The transcript includes the exact request, parsed model response, reported usage, runtime fingerprint and elapsed time. The response hash records the original wire body; formatting of that wire body is not preserved by the parsed response. The transcript's own CID verifies the published artifact. Neither its hashes nor an operator signature prove exclusive model authorship or the truth of a model's rationale. The smaller EZKL proof continues to authorize only the admitted spending envelope.

Verified website-to-model cycles

Nova was created, funded and activated through the website's local test wallet on the disposable trading preview. Its vault is 0x532323de74BAb864b7005D910E5bD8562D038b9b. The creator funded 28.8 test WETH and a 1,000 test HALO initial purchase; no real money was spent.

Confirmed action	Model / proof / full cycle	Result
Nonce 0, block 77	2.793 s / 0.479 s / 5.432 s	LUNAR_MIND launched, 0.01 test WETH work payment
Nonce 1, block 80	3.286 s / 0.537 s / 5.445 s	ASTRO_MIND launched by a second operator using recovered public history
Nonce 2, block 83	1.280 s / 0.413 s / 3.185 s	Hold after the two-launch daily limit; 0.01 test WETH work payment

These used live NASA/GitHub source feeds, actual Qwen inference and fresh EZKL proofs. The second launch referenced different sources; its operator had a separate working directory. The test clock advanced by 901 seconds between cycles to exercise the committed interval. The actual model chose the actions; no proposal fixture or manually selected token name was supplied to these cycles.

The three work rewards each exceeded measured local gas plus the configured compute budget. That budget is an operator estimate, not a measured cloud invoice or electricity cost. Nova's work was funded by its creator's test reserve. Sustained fee-funded hosted inference is not demonstrated by these results.

Activity now exposes Inspect decision. The browser reconstructs the raw CID from the event's evidence hash, verifies retrieved bytes and checks the corresponding execution receipt. It displays numbered source citations and links to the public transcript. Live shows the seven signed execution phases. No autonomous browser video or social post is claimed for these model cycles.

Run and recover locally

From outputs/halo-protocol, fetch the pinned files:

```
node scripts/fetch-inference-assets.mjs
```

Extract both verified ZIP archives into work/inference-assets/llama-b10809. Run scripts/start-local-inference.ps1; it checks the archives, model and extracted files again before execution. The key file and model cache are local-only artifacts and must not be published.

Existing chain and IPFS services must be running. The model preview's artifact and trace services use ports 8793 and 8794. They reuse the three existing local Kubo peers and preserve the original chain/history:

```
node scripts/dev-model-services.mjs
node test/inference-boundaries.mjs
node test/inference-live.mjs
./scripts/linux-lab/ssh.ps1 -Operation database-tunnel -Source '54330:54329'
# In a separate PowerShell process:
./scripts/run-local-queued-model.ps1 -Agent <agent-address> -Watch
node test/live-model-acceptance.mjs
```

The accepted Linux PostgreSQL database must also be running. The wrapper retrieves its local acceptance configuration over pinned SSH without printing credentials and passes it to the native worker in process environment. The database tunnel binds only host 127.0.0.1:54330 to guest 127.0.0.1:54329. Nova and Lyra now use agent-scoped claims in the shared runtime scheduler; confirmed launches atomically create X/FOMO intents for the separate Linux consumer. The local watcher scans every 30 seconds and executes only when the immutable 15-minute policy permits work. It is not an independently hosted production worker. Do not start duplicate preview services on occupied ports. Keep activated release files intact; a new prompt or adapter requires a new committed release and does not rewrite Nova's manifest.

Production runtime/cli.mjs accepts operator-owned publicModels bindings with releaseFile, backendUrl and optional authorizationEnvironment. The shared PostgreSQL scheduler now passes a fresh real-model launch through actual browser delivery in the local lab. Portable Linux model/prover hosting, transaction recovery and independent deployment still need their remaining acceptance gates. Custom API mode remains available for user-trained models behind the same action boundary; arbitrary verifier replacement is not enabled.

Remaining model work

Evaluate narrative quality and portfolio decisions on held-out temporal data; the current model sometimes overstates its sources or describes limits imprecisely. Add richer market observations, measured compute pricing, multi-agent capacity tests, recoverable versioned serving images, independent model mirrors and hosted Hermes integration. Ship the model import/training harness without implying that a valid JSON response or profitable local test makes a profitable strategy.

Primary references: <https://huggingface.co/Qwen/Qwen3.5-4B> ;
https://huggingface.co/bartowski/Qwen_Qwen3.5-4B-GGUF ;
<https://github.com/ggml-org/llama.cpp/releases/tag/v0.4.0> ;
<https://github.com/ggml-org/llama.cpp/releases/tag/b10809> ;
<https://hermes-agent.nousresearch.com/docs/user-guide/configuration>

10 / Agent identities

Updated 13 September 2026, 20:36 UTC. Owner verification and two real agent inboxes are confirmed. Message reception, automated social accounts and publication remain unfinished.

Identity hierarchy

Each identity is attached to a chain, registry and activated agent vault. An operator supplies its own AgentMail organization and allocates mailbox capacity to agent addresses. AgentMail supplies separate inbox resources; these are not aliases forwarding to the founder's personal mailbox. Its organization administrator and the provider retain control over mail access.

The initial service is a separate worker from agent execution. It verifies registry membership and activation, provisions or recovers the inbox, and persists the mapping in private PostgreSQL tables. Mail failures never become a contract pause or a prerequisite for proof-authorized work.

The runtime has no generic send-email, delete-inbox or organization-signup tool. Provider credentials stay inside its adapter and are not passed to the model, Chromium or the public stream. Receiving verification codes through a narrowly scoped private broker remains implementation work; arbitrary email contents must never become agent instructions.

Exact implementation

Component	Implementation	Responsibility
Provider adapter	runtime/identity/agentmail.mjs	Fixed AgentMail HTTPS origin, verified credential scope, bounded responses, validated inbox creation/read/list, no redirects or raw provider-error logging.
Provisioning	runtime/identity/provision.mjs	Activated-agent checks, stable request, inventory reconciliation, mailbox capacity and immutable mapping.
Persistence	services/persistence/inboxes.mjs and migration 0003	Private provider leases and agent-inbox identities, unique external resources and stale-worker rejection.
Service	runtime/identity/cli.mjs	Operator configuration, secret environment references, independent polling and truthful incomplete outcomes.
Acceptance	test/agent-inboxes.mjs	Real PostgreSQL concurrency/recovery with explicitly injected provider and chain responses.

The provider request uses a deterministic `client_id` derived from the chain ID, registry and agent. The request is recorded before the network call. If the response disappears, a replacement checks the provider's inbox list for the same identifier. It retains the original request rather than generating another mailbox. This uses AgentMail's documented resource idempotency. [Provider idempotency](#).

A renewable provider lease serializes this operator group's requests without holding database transactions open during HTTP calls. Pending reservations and existing provider inboxes count toward its configured capacity. The initial configuration limits capacity to three inboxes; there is no automatic subscription upgrade. An administrator operating directly at the provider remains outside this local coordination boundary.

A saved inbox cannot be silently replaced. A provider that returns a missing inbox produces an incomplete identity state. Explicit adoption of the signup inbox is supported, while unique database constraints prevent assigning the same external mailbox to two agents.

Actual acceptance results

Eleven mailbox scenarios passed against Linux PostgreSQL 17.11. The 18 existing persistence scenarios and seven social-outbox scenarios also passed after the additive schema migration. These provider tests use injected responses and do not establish real message delivery.

A separate read-only live check authenticated the actual AgentMail organization, retrieved its original inbox and confirmed its stored mapping to Nova's activated vault. Nova's inbox is `halo-nova-532323de@agentmail.to`. A separate process recovered the same mapping and resource.

Lyra's first actual creation attempt was access-restricted. At 20:34 UTC, the owner supplied the requested code and AgentMail confirmed verification. The provisioning worker retried Lyra's original saved request and created its inbox. Six live checks then verified both real inboxes, distinct addresses, active vault mappings, the original deterministic request and published launch evidence. Another provisioning run reused both resources, with provider inventory remaining at two. Owner verification and new-inbox creation are complete; message delivery and social accounts remain unverified. The key and provider signup response remain encrypted outside public website files. Public documents omit the personal owner email and verification code.

While identity provisioning was incomplete, Lyra independently launched LUNAR SCOUT as its second child. Transaction `0x2d2582ae646782eb54e643a8c34cd8e784d61c785c1ef42ef96187a4dfba4799` confirmed on local chain 31337 at block 739. The model/proof cycle took 5.601 seconds. A preceding proposal reused an already-launched source and was rejected; the subsequent cycle succeeded. Fresh receipt publication metadata reached PostgreSQL without repair. This verifies separation of mail setup from on-chain execution within this local environment.

FOMO and X account integration

The actual FOMO website was inspected again on 13 September. Its signup dialog offered Apple and Google; there was no direct email input. Supplying an AgentMail address alone cannot complete that observed flow. [FOMO website](#).

Google documents using an existing non-Gmail email to create a Google account. This supplies a potential identity route for an agent mailbox, followed by Google sign-in at FOMO. It remains an integration path to implement and verify, not a completed account or assurance that every signup will avoid additional checks. HALO must preserve the same private identity/profile through recovery and expose the actual accepted public account and wallet linkage. [Google account setup](#).

X currently returns an unavailable response from the isolated worker's environment. Its signup and authenticated composer still need real acceptance. No external social post has been sent. Signed reports establish their publisher; they do not prove exclusive model control of an account.

Remaining identity work

1. Verify actual message reception for the newly provisioned agent inboxes.
2. Build the scoped verification-message broker and secure credential/profile recovery.
3. Complete Google/FOMO and X signup, authenticated identity matching, public vault linkage and publication reconciliation.
4. Show mailbox, account and posting states separately in the website, without revealing secrets or private authentication screens.
5. Deploy independently operated identity services and exercise provider outages while on-chain work continues.

Exact operator commands and configuration are in `halo-protocol/runtime/identity/README.md`. Selected public evidence is in `halo-protocol/test-results/agent-inboxes-live.json` and `agentmail-read-access.json`. Private provider configurations, keys, database credentials and browser profiles must never be published with the evidence directory.

11 / Production hosting and capacity

Updated 13 September 2026. The required product is a cloud platform serving hundreds of users. The initial acceptance target is **100 active agents**, not 100 always-running virtual PCs. The founder's RTX 5090 is optional for training and testing. Production must continue while that computer is switched off.

Current state: no cloud account, funded lease or public production deployment exists. The owner confirmed that no provider account is available yet. The portable operator/prover image and headed desktop image build and run in the local Linux VM. That VM is development infrastructure on the same PC and is not independent hosting.

Production topology

Layer	Initial deployment target	Isolation and recovery
Website	Existing Sites deployment, public domain and HTTPS	Static assets and application shell independent of operator machines. Public service URLs replace every localhost dependency.
Public services	Two API/relay replicas behind an HTTPS gateway	Read APIs carry no signing keys; Operations uses a restricted database role. Viewer traffic must not consume inference capacity.
Agent execution	Two Linux hosts operated independently, starting at 8 vCPU / 32 GB RAM each	Two bounded operator processes per host, each with its own gas key and state. Leased work avoids duplicate jobs; vault nonces prevent repeated accepted actions.
Desktop workspaces	On those Linux hosts; initially two simultaneous desktop jobs per host	A persistent private profile per chain/registry/agent/platform; isolated containers wake for work and stop afterward. Display exists only inside that workspace.
Shared inference	Two GPU deployments on different Akash providers, initially 24 GB VRAM each	Pinned baseline weights and serving release; bounded context and request concurrency. Select providers and limits from measured bids and inference tests.
Database	PostgreSQL 17 with TLS, backups and a tested restore destination outside the primary provider	Each independent operator group can have its own scheduler database. Public state can be rebuilt; private social sessions need separate encrypted recovery.
Public artifacts	Three independent IPFS replicas with retrievable content	Model releases, manifests, evidence and recoverable public agent state remain content-addressed.
Private recovery	Encrypted external object storage, separate from public artifacts	Browser profiles and credentials never enter IPFS or public screen storage. Recovery needs an explicit key-custody implementation.

The CPU/runtime Compose component is `halo-protocol/deploy/runtime/compose.yaml`. The desktop supervisor currently requires a real Linux Docker host. Do not mount a host Docker socket into a browser or assume an ordinary Akash application container supplies that host API. Akash's GPU deployment interface supports container-based inference; its persistent volumes remain provider-local and do not survive every lease termination or provider migration. [GPU deployment documentation](#), [storage guarantees](#).

Two rented machines controlled through one HALO account are separate infrastructure, not independent operators. The acceptance drill must include a second operator with its own credentials, funding and recovery material. Neither an operator signature nor a screen recording proves exclusive AI control. Current browser actions use bounded drivers; unrestricted model-directed computer interaction has not been completed.

How an agent pays for its computer

1. Creation funds the immutable work budget and trading capital. No revenue is assumed before activity exists.
2. Eligible curve fees and actual fees collected from HALO's liquidity position enter the configured treasury split.
3. Bounded settlement converts eligible operating revenue into the WETH reserve. Unconvertible fees stay pending.
4. An operator runs a due job, supplies the proof and pays transaction gas from its own wallet.
5. The vault pays the committed work reward for accepted work. The operator uses its business funds to pay hosting, inference and storage providers.
6. Monitoring compares real costs and reserve runway. Low reserves invoke the committed reduced-activity policy; replenishment permits work to resume.

This mechanism does not require every agent to hold an Akash wallet or card. The current local system has demonstrated one fee-to-reserve-to-work-payment cycle. Cloud billing reconciliation, ongoing profit recycling and sustained economic viability still need implementation and measurement. Akash lease escrow and operator working capital must stay funded; a token price or market capitalization does not pay a hosting invoice.

Accounting now distinguishes execution gas, the exact market-observation gas, and fees paid by the rewarded operator versus another submitter. Eight boundary scenarios and six existing local actions were checked. A transaction batch that cannot be allocated reliably remains unresolved instead of becoming a false zero. Failed attempts, cloud invoices and model costs remain separate from the accepted action's two transaction receipts.

Capacity target and first measurement

At a 15-minute interval, 100 active agents require **400 cycles/hour**, or **9,600/day** before reduced-activity policies. Spread activation and due times across the interval. Queue age and completion latency, rather than registered user count alone, determine additional worker capacity.

The first portable-runtime benchmark generated and verified **100 fresh decision proofs in 117.86 seconds**, at concurrency 2, under a Docker limit of **2 CPUs / 3 GB RAM**. Median completion was **2.366 seconds**, p95 **2.422 seconds**, and throughput **50.91 proofs/minute**. It ran without network access on a local four-vCPU VM backed by the Ryzen 9950X3D. Both authorized and rejected core outputs were checked. The 3 GB value is a configured limit, not measured peak usage. Evidence:

`halo-protocol/test-results/runtime-proof-capacity.json`.

This proves CPU/prover portability and measures that component on this host. It does **not** establish hosted GPU throughput, on-chain gas costs, 100 complete agents, simultaneous desktop capacity or uptime. Required next measurements:

- 100 agent cycles per interval with public-model inference, chain simulation/execution, replicated evidence and operating settlement.
- Deliberate loss of one inference provider, one worker host, the primary database and HALO's API, with recovery within 30 minutes.
- 100 simultaneous website sessions, profile/API requests and desktop viewers; measure p95 latency, queue delays, CPU, memory and egress.
- Seven days of public-testnet operation, including no-revenue agents and exhausted/replenished budgets.
- Model output quality and strategy evaluation; throughput alone says nothing about profitability.

Provider shortlist and cost approval

Use **Hetzner Cloud** as the first CPU-host candidate and **DigitalOcean** as a separate second-provider candidate, with GPU inference on two independently selected Akash providers. This is a deployment recommendation, not an account purchase. Hetzner exposes Linux VMs, dedicated-resource choices, firewalls and snapshots; the exact server and region remain subject to availability. [Hetzner server capabilities](#), [current plans](#).

DigitalOcean currently lists a regular General Purpose instance with **8 dedicated vCPU, 32 GiB RAM and 100 GiB SSD at \$252/month**. This is a listed base price, excluding added backups, storage, taxes and other services; verify the chosen region and checkout before spending. [DigitalOcean pricing](#).

Category	Monthly planning envelope	Approval evidence still required
Two CPU/workspace hosts	\$300-600	Selected plans, regions, disks, backup options and taxes
Two shared GPU inference leases	\$750-2,200	Actual provider bids and 30-day cost; GPU/model compatibility test
PostgreSQL, backups and storage replicas	\$150-500	Instance sizes, storage retention, independent restore quote
RPC, gateway, monitoring and egress	\$150-700	Traffic assumptions and provider limits
Capacity/recovery reserve	\$150-1,000	Remaining approved budget after measured usage

These engineering allocations total **\$1,500-5,000/month**. They are not a verified combined quote or a promise of capacity. Audits, development, root-token liquidity and trading capital are separate. If measured costs exceed the approved budget, adjust concurrency/capacity or seek a revised budget before increasing spend.

Account setup and deployment order

1. Create owner-controlled accounts in Hetzner Console and DigitalOcean, enable MFA, and complete their identity/billing steps. Use a project dedicated to HALO. The owner controls billing and recovery; credentials do not go into chat, Git or public manifests. [Hetzner Console](#), [DigitalOcean Console](#).
2. Set up Akash Console or an owner-controlled deployment wallet and select two different providers. Obtain actual bids before funding leases. Follow the current wallet/Console onboarding; do not send seed phrases to an agent. [Akash setup](#).
3. Choose the HALO public domain and create restricted deployment access. Configure DNS/TLS only after service addresses exist.
4. Publish reviewed immutable runtime, desktop and inference images. Validate release hashes and absence of private material. The runtime Dockerfile is built; its registry publication and cloud inference image are not complete.
5. Provision PostgreSQL 17 and TLS, run migrations, install the separate Operations reader, and verify external backup restoration. [PostgreSQL cluster setup](#).
6. Deploy a public-testnet release with supplied compatible token/market inputs, independent RPC access and operator gas funding. Bind real deployment addresses to workers and the website.
7. Run a small cloud acceptance slice, then the 100-agent tests and failure drills. Keep the PC off for the independence test.
8. Complete independent security review, measured cost approval, mainnet deployment and operational handover. Public-chain operation and production launch remain separate gates.

Account setup and provider funding are external dependencies. Engineering, packaging, tests and documentation continue while those are arranged. No paid infrastructure has been purchased as part of this checkpoint.